

Llenguatges de Programació

7 de gener de 2026

Grau en Enginyeria Informàtica

Gerard Escudero, Jordi Petit
Facultat d'Informàtica de Barcelona
Universitat Politècnica de Catalunya

© 2026 Universitat Politècnica de Catalunya, 2026 (UPC). Tots els drets reservats. Aquest examen no pot ser publicat, compartit, reproduït o distribuït sense el consentiment exprés per escrit de la UPC.

Instruccions

L'examen dura dues hores. Poseu el vostre nom a cada full. Contesteu cada problema en fulls diferents. Es valorarà la concisió, claredat i brevetat a més de la completesa i l'exactitud de les respostes. Feu bona lletra.

1 λ -càlcul

(2 punts)

Recordeu que en λ -càlcul:

$$\begin{aligned} 2 &\equiv \lambda s.\lambda z.s(sz) \\ \mathbf{succ} &\equiv \lambda n.\lambda s.\lambda z.s(nsz) \\ \mathbf{add} &\equiv \lambda m.\lambda n.\lambda s.\lambda z.ms(nsz) \end{aligned}$$

1. Avalueu en ordre normal:

succ (add 2 3)

2. Penseu en el significat de les variables m i n de les definicions anteriors. Quina funció pot estar representant el combinador λ -càlcul següent?

$$f = \lambda x.\lambda s.\lambda z.s(xs(xsz))$$

2 Factorials

(5 punts)

- 1a) Inferiu el tipus de la funció *fix* definida per:

$$\text{fix } f = f (\text{fix } f)$$

Per a fer-ho, dibuixeu l'arbre de sintaxi abstracta corresponent, anoteu l'arbre amb els tipus de cada node, genereu el conjunt de restriccions d'igualtat de tipus i resoleu les restriccions. No us deixeu els requeriments sobre les classes si n'hi ha.

- 1b) Utilitzant la funció *fix* i sense utilitzar recursivitat explícita, definiu la funció *fact1* que calcula el factorial d'un enter no negatiu:

$$\begin{aligned} \text{fact1} &:: \text{Integer} \rightarrow \text{Integer} \\ \text{fact1 } n &= \dots \end{aligned}$$

- 2a) Expliqueu (en una frase) què és el Continuation-Passing Style (CPS).

- 2b) Escriviu una solució per calcular el factorial d'un número amb la funció *fact2* utilitzant CPS.

$$\begin{aligned} \text{fact2} &:: \text{Integer} \rightarrow \text{Integer} \\ \text{fact2 } n &= \dots \end{aligned}$$

- 3a) Inferiu el tipus de la funció *trampoline* donada per aquestes definicions:

$$\text{data Trampoline } a = \text{Done } a \mid \text{More (Trampoline } a)$$

$$\begin{aligned} \text{trampoline (Done } x) &= x \\ \text{trampoline (More } t) &= \text{trampoline } t \end{aligned}$$

- 3b) Inferiu el tipus de *fact3T* dins d'aquesta tercera funció per calcular el factorial d'un número utilitzant la tècnica del trampolí:

$$\begin{aligned} \text{fact3} &:: \text{Integer} \rightarrow \text{Integer} \\ \text{fact3 } n &= \text{trampoline (fact3T 1 } n) \end{aligned}$$

Per a fer-ho, dibuixeu l'arbre de sintaxi abstracta corresponent, anoteu l'arbre amb els tipus de cada node, genereu el conjunt de restriccions d'igualtat de tipus i resoleu les restriccions. No us deixeu els requeriments sobre les classes si n'hi ha.

- 3c) Implementeu *fact3T*.

3 foldM i closures

(3 punts)

1. La documentació de Haskell descriu la funció **foldM** com:

The foldM function is analogous to foldl, except that its result is encapsulated in a monad.

```
foldM :: (Foldable t, Monad m) => (b -> a -> m b) -> b -> t a -> m b
foldM f a1 [x1, x2, ..., xm]
==
do
  a2 <- f a1 x1
  a3 <- f a2 x2
  ...
  f am xm
```

Implementeu amb **foldM** una funció `sumUntil :: [Int] -> Either Int Int` que doni la suma dels elements de la llista si no hi ha cap nombre negatiu. En cas de trobar un nombre negatiu haurà d'aturar el recorregut i mostrar-lo.

```
ghci> sumUntil [1..5]
Right 15
ghci> sumUntil [1,-1,2,-3,5]
Left (-1)
```

2. Donat el codi següent:

```
import random
...
@retry
def f():
    pr = random.random()
    if pr < 0.7:
        raise Exception("ko: " + str(pr))
    return "ok: " + str(pr)
print(f())
```

Implementeu el decorador `retry` per a que vagi cridant a la funció fins que no salti cap excepció. A continuació teniu un exemple de sortida:

```
ko: 0.12350700795970082
ko: 0.4897773801102827
ko: 0.08681657809832988
ko: 0.036684967357044185
ok: 0.8133782982490444
```

Per capturar un excepció podeu fer:

```
try:
    # codi
except Exception as e:
    print(e)
```

3. Representeu (de forma semblant com ho faria Python Tutor) l'estat de la memòria després de l'execució de l'última línia d'aquest programa:

```
def mf(x):  
    z = x[:]  
  
    def f(y):  
        return sum(x) + sum(z) + y  
  
    return f
```

```
a = [1, 2, 3]  
b = a  
g = mf(a)  
i = g(10)  
b.append(4)  
j = g(10)
```

A continuació teniu un exemple d'execució del Python Tutor:

